

Deploy da Immagine Container

Modulo: 3 — Deploy di Applicazioni

Versione: OpenShift 4.14+ su Azure ARO

Obiettivi di Apprendimento

- Deployare un'applicazione su OpenShift a partire da un'immagine container esistente con `oc new-app`
- Comprendere le risorse create automaticamente da `oc new-app` (Deployment, Service, ImageStream)
- Esporre un'applicazione all'esterno con `oc expose` e verificarne l'accessibilità tramite Route
- Distinguere `oc new-app` da `oc create deployment` e scegliere il comando appropriato
- Aggiornare l'immagine di un Deployment esistente e monitorare il rollout
- Configurare correttamente `imagePullPolicy` per ambienti di sviluppo e produzione

1. Teoria e Concetti Chiave

Come OpenShift Gestisce le Immagini

OpenShift introduce il concetto di **ImageStream** (flusso di immagini), un livello di astrazione sopra le immagini container che permette di:

- Tracciare le versioni di un'immagine tramite **tag** (es. `latest`, `v1.0`, `stable`)
- Triggerare automaticamente un nuovo rollout quando l'immagine sottostante cambia
- Referenziare immagini da registry esterni (Quay.io, Docker Hub, ACR) in modo uniforme

Registry Esterno		ImageStream OCP		Deployment
quay.io/myapp:1.0	→	myapp:1.0 (tag)	→	spec.image: myapp:1.0
quay.io/myapp:1.1	→	myapp:1.1 (tag)	→	trigger → nuovo rollout

oc new-app: Il Comando Tuttotfare

`oc new-app` è il comando principale per creare applicazioni in OCP. Quando riceve un'immagine container come argomento, crea automaticamente:

Risorsa	Descrizione
Deployment	Gestisce i Pod dell'applicazione
Service	Espone l'applicazione internamente al cluster (ClusterIP)
ImageStream	Traccia l'immagine sorgente e i suoi aggiornamenti

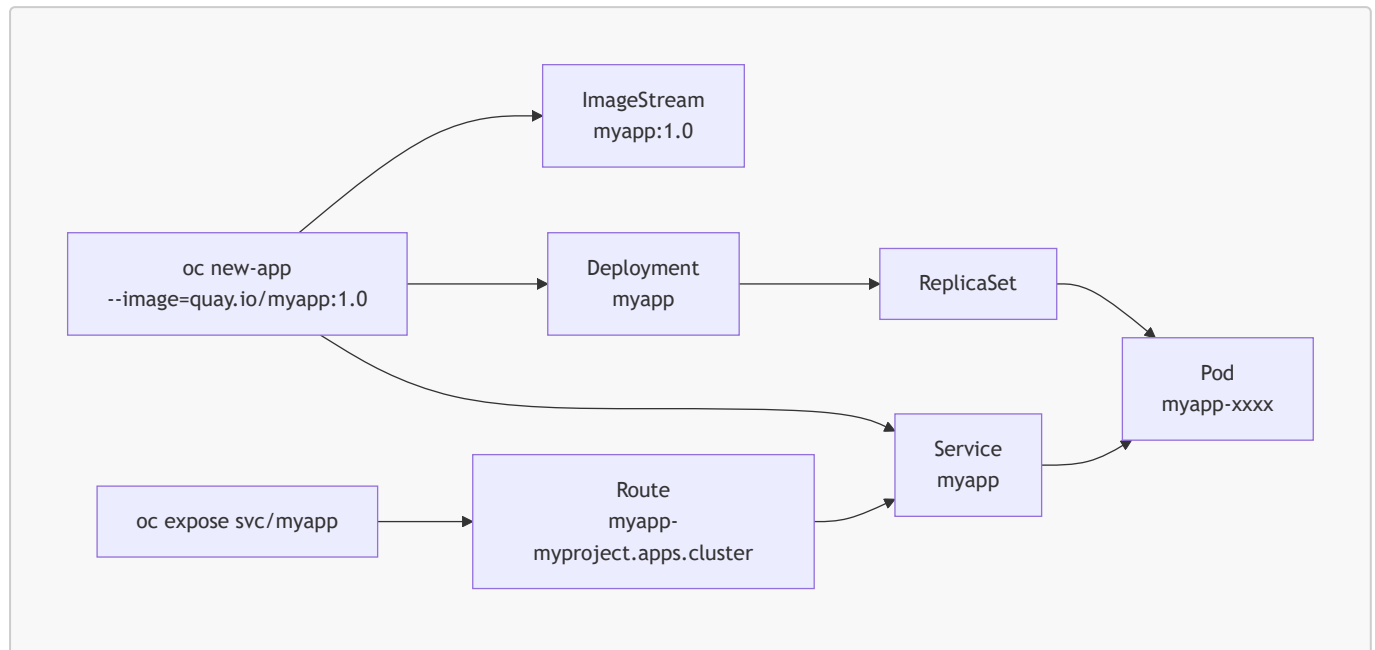
 **Suggerimento:** `oc new-app` non crea una `Route` automaticamente. Bisogna eseguire `oc expose service/<nome>` separatamente per esporre l'applicazione all'esterno.

oc create deployment vs oc new-app

Caratteristica	oc new-app	oc create deployment
Crea Service	✓ Automatico	✗ Manuale
Crea ImageStream	✓ Automatico	✗ No
Trigger aggiornamento immagine	✓ Sì (via ImageStream)	✗ No
Rilevamento automatico	✓ Linguaggio/framework	✗ Solo immagine
Uso consigliato	Deploy rapido, sviluppo	Deploy controllato, produzione
Output YAML	Con <code>--dry-run -o yaml</code>	Con <code>--dry-run=client -o yaml</code>

2. Diagramma — Flusso di Deploy da Immagine

Il seguente diagramma mostra il percorso completo: dall'esecuzione del comando al Pod in esecuzione e alla Route accessibile dall'esterno.



3. Comandi Pratici con Output Atteso

```

# Deployare un'applicazione da immagine pubblica su Quay.io
oc new-app --image=quay.io/bitnami/nginx:latest --name=web-nginx -n my-project

# Output atteso:
# --> Found container image a1b2c3d (2 weeks old) from quay.io for
"quay.io/bitnami/nginx:latest"
# --> Creating resources ...
#   imagestream.image.openshift.io "web-nginx" created
#   deployment.apps "web-nginx" created
#   service "web-nginx" created
# --> Success
#   Application is not exposed. You can expose services to the outside world by
executing one or more of the commands below:
#   'oc expose service/web-nginx'
#   Run 'oc status' to view your app.

# Verificare le risorse create
oc get all -n my-project -l app=web-nginx

# Output atteso:
# NAME                                     READY   STATUS    RESTARTS   AGE
# pod/web-nginx-7d9f8b6c4-xk2p9          1/1     Running   0           45s
#
# NAME                TYPE          CLUSTER-IP    EXTERNAL-IP   PORT(S)    AGE
# service/web-nginx   ClusterIP     172.30.45.123 <none>        80/TCP      45s

```

```
#
# NAME                                READY    UP-TO-DATE    AVAILABLE    AGE
# deployment.apps/web-nginx           1/1      1              1            45s
#
# NAME                                DESIRED    CURRENT    READY    AGE
# replicaset.apps/web-nginx-7d9f8b6c4 1          1          1        45s
#
# NAME                                IMAGE REPOSITORY
TAGS      UPDATED
# imagestream.image.openshift.io/web-nginx image-registry.openshift-image-
registry.svc:5000/... latest 45s

# Esporre il Service creando una Route HTTP
oc expose service/web-nginx -n my-project

# Output atteso:
# route.route.openshift.io/web-nginx exposed

# Visualizzare l'URL della Route
oc get route web-nginx -n my-project

# Output atteso:
# NAME          HOST/PORT          PATH    SERVICES
PORT          TERMINATION    WILDCARD
# web-nginx     web-nginx-my-project.apps.cluster.example.com    web-nginx
8080-tcp      None

# Verificare lo stato del rollout
oc rollout status deployment/web-nginx -n my-project

# Output atteso:
# deployment "web-nginx" successfully rolled out

# Visualizzare i log del Pod
oc logs deployment/web-nginx -n my-project

# Seguire i log in tempo reale
oc logs -f deployment/web-nginx -n my-project

# Visualizzare gli eventi del namespace per diagnostica
oc get events -n my-project --sort-by='.lastTimestamp'

# Deploy con oc create deployment (approccio imperativo Kubernetes)
oc create deployment my-app \
  --image=registry.access.redhat.com/ubi9/ubi-minimal:latest \
  --replicas=2 \
  -n my-project

# Output atteso:
# deployment.apps/my-app created

# Generare il YAML senza applicarlo (dry-run – utile per GitOps)
oc new-app --image=quay.io/bitnami/nginx:latest --name=web-nginx \
  --dry-run=true -o yaml -n my-project
```

```
# Aggiornare l'immagine di un Deployment esistente
oc set image deployment/web-nginx \
  web-nginx=quay.io/bitnami/nginx:1.25 \
  -n my-project

# Output atteso:
# deployment.apps/web-nginx image updated

# Verificare che il rollout sia avvenuto
oc rollout status deployment/web-nginx -n my-project
# Output atteso:
# Waiting for deployment "web-nginx" rollout to finish: 1 out of 1 new replicas
# have been updated...
# deployment "web-nginx" successfully rolled out
```

4. Esempi YAML Completi

Deployment completo con imagePullPolicy

```
# file: deployment-web-nginx.yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: web-nginx
  namespace: my-project
  labels:
    app: web-nginx          # label per identificare l'applicazione
    version: "1.25"
spec:
  replicas: 2               # numero di Pod desiderati
  selector:
    matchLabels:
      app: web-nginx        # deve corrispondere ai label del template Pod
  strategy:
    type: RollingUpdate
    rollingUpdate:
      maxSurge: 1           # Pod extra durante l'aggiornamento
      maxUnavailable: 0    # nessun Pod non disponibile durante l'aggiornamento
  template:
    metadata:
      labels:
        app: web-nginx
        version: "1.25"
    spec:
      containers:
        - name: web-nginx
          image: quay.io/bitnami/nginx:1.25
          imagePullPolicy: Always # scarica sempre l'immagine (consigliato per
:latest)
```

```

ports:
- containerPort: 8080
  protocol: TCP
resources:
  requests:
    cpu: "100m"
    memory: "128Mi"
  limits:
    cpu: "500m"
    memory: "256Mi"
readinessProbe:          # Pod pronto solo quando risponde su /
  httpGet:
    path: /
    port: 8080
    initialDelaySeconds: 5
    periodSeconds: 10
livenessProbe:           # riavvia il Pod se non risponde
  httpGet:
    path: /
    port: 8080
    initialDelaySeconds: 15
    periodSeconds: 20

```

```

# Applicare il Deployment
oc apply -f deployment-web-nginx.yaml

```

Service per esporre il Deployment

```

# file: service-web-nginx.yaml
apiVersion: v1
kind: Service
metadata:
  name: web-nginx
  namespace: my-project
  labels:
    app: web-nginx
spec:
  selector:
    app: web-nginx          # seleziona i Pod con questo label
  ports:
  - name: http
    port: 80                # porta del Service (interna al cluster)
    targetPort: 8080        # porta del container
    protocol: TCP
  type: ClusterIP           # solo interno al cluster

```

Route per esporre all'esterno

```
# file: route-web-nginx.yaml
apiVersion: route.openshift.io/v1
kind: Route
metadata:
  name: web-nginx
  namespace: my-project
  labels:
    app: web-nginx
spec:
  to:
    kind: Service
    name: web-nginx
    weight: 100
  port:
    targetPort: http          # nome della porta nel Service
  tls:
    termination: edge         # TLS terminato al router OCP (HTTPS → HTTP verso il
Pod)
    insecureEdgeTerminationPolicy: Redirect # reindirizza HTTP → HTTPS
    wildcardPolicy: None
```

5. Note Specifiche per Azure ARO

```
# Su ARO, per usare immagini da Azure Container Registry (ACR) privato,
# bisogna creare un Secret con le credenziali ACR e linkarlo al ServiceAccount

# 1. Creare il Secret con le credenziali ACR
oc create secret docker-registry acr-secret \
  --docker-server=myregistry.azurecr.io \
  --docker-username=myregistry \
  --docker-password=$(az acr credential show --name myregistry --query
passwords[0].value -o tsv) \
  -n my-project

# 2. Linkare il Secret al ServiceAccount 'default' del namespace
oc secrets link default acr-secret --for=pull -n my-project

# 3. Deploy da ACR
oc new-app --image=myregistry.azurecr.io/myapp:latest --name=myapp -n my-project
```

⚠ Attenzione ARO: Su ARO il registry interno di OCP (`image-registry.openshift-image-registry.svc`) è gestito da Azure e usa Azure Blob Storage come backend. Non modificare la configurazione del registry interno.

Aspetto	OCP On-Premise	Azure ARO
Registry interno	Configurato dall'admin	Gestito da Microsoft/Red Hat
Registry privato	Qualsiasi	Consigliato Azure ACR
imagePullSecret	Opzionale per registry pubblici	Obbligatorio per ACR privato
SCC per i Pod	<code>restricted-v2</code> default	<code>restricted-v2</code> default (più restrittivo)

⚠ **Attenzione SCC:** Su ARO (come su OCP 4.11+) il SCC default è `restricted-v2` che vieta l'esecuzione come root. Se l'immagine richiede root, usare un'immagine alternativa o richiedere all'admin un SCC appropriato.

6. Riepilogo dei Punti Chiave

- `oc new-app --image=` crea automaticamente **Deployment + Service + ImageStream**; la Route va creata manualmente con `oc expose`
- L'**ImageStream** traccia l'immagine sorgente e può triggerare rollout automatici quando l'immagine cambia
- `oc create deployment` crea solo il Deployment (approccio Kubernetes standard), senza Service né ImageStream
- `imagePullPolicy: Always` è consigliato per tag `latest`; `IfNotPresent` è preferibile per tag versione in produzione
- `oc set image deployment/<nome> <container>=<nuova-immagine>` aggiorna l'immagine senza modificare il YAML
- Su ARO, le immagini da **Azure Container Registry (ACR) privato** richiedono un `docker-registry` Secret linkato al ServiceAccount
- Il SCC `restricted-v2` su ARO impedisce l'esecuzione come root: usare sempre immagini non-root

7. Domande di Verifica

1. Quale risorsa OpenShift NON viene creata automaticamente da `oc new-app --image=?`

- A) Deployment
- B) Service
- C) **Route** ✓
- D) ImageStream

2. Quale `imagePullPolicy` è consigliata per un tag versione fisso in produzione (es. `myapp:1.5.2`)?

- A) Always
- B) **IfNotPresent** ✓
- C) Never
- D) OnFailure

3. Come si aggiorna l'immagine di un container in un Deployment esistente senza modificare il file YAML?

- A) `oc new-app --image=<nuova>`
- B) `oc patch deployment`
- C) `oc set image deployment/<nome> <container>=<nuova-immagine>` ✓
- D) `oc replace -f`

4. Su ARO, per fare il pull di un'immagine da un ACR privato, cosa bisogna creare?

- A) Un ConfigMap con le credenziali
- B) **Un Secret di tipo `docker-registry` linkato al ServiceAccount** ✓
- C) Un ClusterRole per il ServiceAccount
- D) Un ImageStreamTag

5. Qual è la differenza principale tra `oc new-app` e `oc create deployment`?

- A) `oc new-app` supporta solo immagini pubbliche
- B) `oc create deployment` crea anche il Service automaticamente
- C) **`oc new-app` crea Service e ImageStream automaticamente; `oc create deployment` crea solo il Deployment** ✓
- D) Non c'è differenza, sono alias dello stesso comando